

SCATTOLINI S.P.A. — STORIA DI INGEGNERIA INTERNA

SSV Manager Roadmap post LinkedIn

Da un memo per l'amministrazione a un sistema unico che governa il ciclo vita del veicolo — dieci post, quattro movimenti narrativi, ogni affermazione ricondotta a un file, un commit o un documento del repository.

Preparato dalla storia del repository [Manager-SSV-](#) (141 commit, 45 pull request unite, 17 marzo – 5 luglio 2026).

Bozza per revisione interna — non per distribuzione esterna così com'è.

SSV Manager — Roadmap dei post LinkedIn

Da un memo per l'amministrazione a un sistema unico che governa il ciclo vita del veicolo in Scattolini S.p.A.

Come è stato costruito questo documento, e come leggerlo

Ogni affermazione tecnica qui sotto è verificabile in `Manager-SSV-` : un file, una funzione, un commit, una PR unita. Dove il repository non dà una risposta verificabile (numeri di utilizzo reali, uptime, quante persone lo usano ogni giorno — tutto ciò che esiste solo in una dashboard Firebase/Vercel o nella memoria di qualcuno), è segnato **[DA CONFERMARE: ...]** invece di essere inventato.

LA TESI CHE ATTRAVERSA TUTTA LA SERIE

Prima di SSV Manager, il ciclo vita di un veicolo allestito in Scattolini viveva diviso in tre sistemi che non si parlavano: un foglio Excel per il tracciamento operativo, email sparse per coordinare officina/collaudo/cliente, e SAP per gli ordini. Nessuno dei tre era il punto di verità: lo stato reale di un veicolo era la somma mentale di "cosa dice l'Excel + cosa ricordo dalle email + cosa c'è su SAP", ricostruita ogni volta da chi doveva prendere una decisione.

SSV Manager prende quei tre pezzi e li fa diventare un sistema unico: uno schema dati condiviso, un solo listener realtime che alimenta ogni pagina, un solo insieme di regole di accesso, automatismi che sostituiscono i passaggi a voce tra reparti. Ogni post di questa serie mostra *quale pezzo specifico* del processo frammentato quella decisione tecnica ha assorbito nel sistema unico — non "cosa ho corretto nel codice", ma "quale coordinamento manuale non serve più".

Due fatti strutturali hanno guidato ogni scelta in questa roadmap:

1. **La storia git non parte da zero.** Il primo commit (`8898367` , 17 marzo 2026) contiene già un `App.jsx` funzionante di 710 righe, un `firestore.rules` con controlli sui ruoli, un workflow di backup, e il README generico di Vite mai sostituito. Ma il repository ha custodito, fino a questa settimana, anche un file chiamato `DOCUMENTO_TECNICO_AMMINISTRAZIONE.md` , datato **23 gennaio 2026**, scritto "per l'amministrazione aziendale", che proponeva questo stesso sistema come sostituto dei processi cartacei e dei fogli Excel — segnalando esplicitamente **l'RBAC come non ancora implementato** e l'MFA come raccomandazione futura. Quel documento è il vero "giorno zero": la storia git è dove la proposta si è chiusa e trasformata in sistema di produzione.
2. **I 141 commit si concentrano in cinque finestre distinte**, non in un flusso costante: 17-19 mar (31 commit, messa in sicurezza prima che il sistema toccasse dati aziendali reali), 25 mar-2 apr (39 commit, un solo punto di verità nel codice + regole di business reali), 23-27 apr (6 commit, rifiniture), 18-22 mag (38 commit, ruoli su misura + migrazione dell'infrastruttura per un vincolo di costo), 3-5 lug — questa settimana — (27 commit, blindatura di concorrenza e pulizia del repository). Quella forma è la

storia: non è "giorno 1, giorno 2, giorno 3", è "la settimana in cui l'abbiamo reso affidabile", "la settimana in cui l'architettura è diventata un solo punto di verità", "la settimana in cui un vincolo di costo ha spinto a ripensare un pezzo di infrastruttura", "la settimana in cui abbiamo chiuso il cerchio".

PERCHÉ QUESTA STRUTTURA (NON UN NUMERO FISSO DI POST)

Non ho fissato un numero di post a priori. Leggendo il repository, dieci punti di svolta reggono da soli — ognuno corrisponde a una PR reale, a un commit reale, o a un documento realmente esistito prima e indipendentemente dagli altri. Una serie più corta avrebbe accorpato commit che non appartengono alla stessa decisione (l'aggiunta del ruolo Omologatore e la migrazione da Cloud Functions sono capitate nella *stessa settimana* di commit ma sono scelte indipendenti, con motivazioni indipendenti); una serie più lunga avrebbe gonfiato commit di routine in post a sé, cosa che il materiale non giustifica. Dieci archi, raggruppati in quattro movimenti narrativi:

- **Movimento I — Prima del codice (1 post):** la proposta che precede il repository.
 - **Movimento II — Dal foglio Excel al primo cancello di fiducia (3 post):** rendere affidabile l'ingresso dei dati, allineare il modello dati al processo reale, portare quel principio dentro la struttura del codice.
 - **Movimento III — Il sistema prende decisioni al posto delle persone (4 post):** la macchina a stati, i ruoli su misura, la migrazione dell'infrastruttura email, l'incidente sui backup.
 - **Movimento IV — Igiene di produzione e chiusura del cerchio (2 post):** un solo criterio di ricerca ovunque, e la pulizia finale del repository come segnale di maturità.
-

Post 1 — Il documento che è nato prima del codice

Movimento I: Prima del codice

Hook: Tre mesi prima del primo commit, qualcuno ha scritto un memo per l'amministrazione di Scattolini con un punteggio di sicurezza di 8/10 e una riga che diceva "RBAC: ⚠ Non implementato." Quel memo è più onesto di molti documenti di prodotto che ho visto, ed è il vero punto di partenza di questa storia.

Obiettivo del post: Stabilire che questo non è "un side project diventato per caso un tool aziendale" — era già uno strumento funzionante, proposto formalmente alla direzione *prima* che il codice in questo repository avesse una storia git, con un elenco onesto di cosa mancava ancora per chiudere il cerchio tra Excel, email e SAP.

Contenuto:

- `DOCUMENTO_TECNICO_AMMINISTRAZIONE.md` (datato 23 gennaio 2026, rimosso dal repository il 5 luglio 2026 nel commit `eb9aba7` come parte di una pulizia finale — il contenuto qui sotto è ricostruito dalla storia git, non da un file ancora presente) descriveva il sistema come già sostitutivo di "processi cartacei e fogli Excel": un'unica piattaforma Firebase + React, 9 moduli funzionali, 30+ campi per veicolo.
- Lo stesso memo lo quantificava: **€0,50-4,50/mese** sul piano a consumo di Firebase — un argomento di costo rivolto esplicitamente all'amministrazione, non a un pubblico tecnico.
- Si autoassegnava un punteggio di sicurezza di 8/10 ed elencava **due gap espliciti**: MFA (non implementato, €0 per aggiungerlo) e **RBAC** (non implementato). Il criterio dietro quella scelta è chiaro: dichiarare cosa manca a chi deve firmare l'adozione del sistema è più utile, nel lungo periodo, di un punteggio gonfiato.
- Incrociando la storia git: un RBAC di base a tre ruoli (`admin` / `operativo` / `non_operativo` , applicato in `firestore.rules` tramite `canWrite()` / `isAdmin()`) era *già presente* nel primissimo commit di questo repository (17 marzo) — il gap segnalato dal memo di gennaio è stato chiuso nelle circa sette settimane tra il memo e il primo commit, un periodo che questo repository non copre.
- **[DA CONFERMARE: chi ha letto/approvato il documento del 23 gennaio, e se la decisione di adozione ufficiale è stata presa in quella data o successivamente]** — il repository mostra l'artefatto, non la riunione.

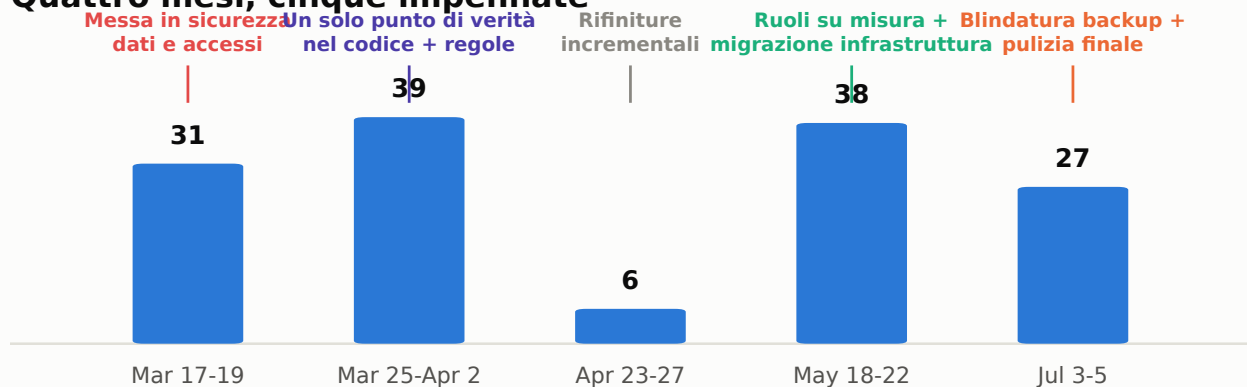
Impatto sul processo aziendale: Prima, la valutazione se adottare o meno lo strumento passava per un documento isolato, senza modo di verificare se le promesse fatte lì (sicurezza, costo, funzionalità) corrispondessero a un sistema realmente in uso. Dopo, quella stessa proposta è verificabile riga per riga nel codice che gira in produzione:

chi doveva decidere aveva un artefatto con un prezzo e un rischio dichiarati, non una demo.

Chiusura: Le proposte interne migliori non nascondono i gap: li prezzano. Qual è l'ultimo documento tecnico che hai scritto ammettendo esplicitamente cosa *non* era ancora pronto?

Visivo: grafico a barre delle cinque finestre di commit che seguono questo memo, per mostrare subito che "adozione" non è stato un evento singolo ma una sequenza di quattro mesi.

Quattro mesi, cinque impennate



commit per periodo attivo (141 commit totali, 45 PR unite)

[SCREENSHOT DA INSERIRE: uno screenshot reale della sezione Executive Summary del documento originale (se recuperabile da uno storage esterno al repository, dato che il file è stato eliminato il 5 luglio), oppure uno screenshot della pagina "Analisi e Statistiche" attuale del sistema come prova visiva che lo strumento descritto nel memo esiste davvero]

Post 2 — Da zero a un cancello di fiducia sui dati

Movimento II: Dal foglio Excel al primo cancello di fiducia

Hook: I primi tre giorni di storia di questo repository non sono feature nuove. Sono la funzione di import CSV che diventa, decisione dopo decisione, il cancello che decide se un dato che arriva da Excel o da SAP può entrare nel punto di verità unico.

Obiettivo del post: Mostrare cosa conteneva davvero il "giorno 1" (17-19 marzo, 31 commit): non nuove funzionalità, ma la decisione deliberata di rendere affidabile il punto in cui i dati storici — sparsi tra fogli Excel distribuiti agli operatori e ordini SAP — entrano per la prima volta in un sistema unico.

Contenuto:

- Commit `7084cec` ("rendi opzionale il numero di telaio + verifica import csv") e `12e1eed` ("rimuovi campi veicolo inutilizzati + pulisci import csv"): il percorso di import (`src/utils/csvUtils.js` → `buildVehicleFromRow`) è stato riverificato campo per campo contro il template Excel reale distribuito agli operatori — criterio: se un campo del template non ha una destinazione precisa nel modello dati, il dato si perde silenziosamente in import, quindi va allineato prima di fidarsi del flusso.
- Commit `842158b`: l'import CSV è stato esteso per accettare `.xlsx` direttamente e per **riconoscere da sé il separatore** (virgola o punto e virgola) — la conseguenza diretta di avere utenti reali che aprono lo stesso file con impostazioni regionali italiane in Excel, un caso che un cancello dati pensato per un solo formato avrebbe ignorato.
- Commit `d772bc7` ("security: rimuove la funzione signup inutilizzata e aggiorna xlsx per correggere CVE-2023-30533"): hardening deliberato della superficie di autenticazione prima di trattare dati aziendali reali. La funzione `signup()` (`createUserWithEmailAndPassword`) restava collegata ad `AuthContext` pur non essendo mai esposta in UI — il criterio applicato è che la superficie di autenticazione va ridotta a ciò che il processo realmente prevede ("gli utenti vengono creati dall'amministratore", come da manuale), non lasciata più ampia di quanto serve. Stesso commit ha portato `xlsx` oltre una CVE nota di prototype pollution.
- Commit `1d93810`: un'intera pagina (`TelaiInAttesaPage.jsx` + `TelaiModal.jsx`, "Telai in Attesa di Ordine") è stata rimossa insieme ai suoi riferimenti nell'esportatore di backup — una decisione di tenere il modello dati allineato al processo reale, non a quello previsto inizialmente: se quel pezzo di flusso non corrisponde più a come Scattolini gestisce l'attesa ordine, tenerlo in piedi significherebbe un secondo punto di verità falso accanto a quello vero.
- Sette pull request (#11-#17), tutte sotto lo stesso prefisso di branch `claude/evaluate-production-readiness-*`, unite in una finestra di 48 ore — il nome del branch dichiara

l'intento in modo più esplicito di qualunque messaggio di commit: prima di trattare dati aziendali reali, verificare sistematicamente che il sistema regga.

Impatto sul processo aziendale: Prima, un file Excel malformato o un campo interpretato in modo diverso da un operatore all'altro produceva un errore silenzioso: il dato semplicemente non entrava, o entrava sbagliato, senza segnale. Dopo, l'import diventa un cancello esplicito: o il dato è conforme allo schema condiviso ed entra nel punto di verità unico, o l'operatore vede subito quali righe non sono valide — l'errore si scopre all'importazione, non tre settimane dopo in officina.

Chiusura: Nessuno programma uno sprint di "verifica di produzione" dopo che una funzionalità è finita — lo si programma quando si decide di trattare per la prima volta i dati reali di un'azienda con quel codice.

Visivo: nessun diagramma generato — questo post è servito meglio da [SCREENSHOT DA INSERIRE: la finestra di importazione CSV/XLSX (`ImportCSVModal.jsx`) con l'anteprima di validazione riga-per-riga, per mostrare visivamente cosa vede l'operatore quando un import ha errori].

Post 3 — Allineare il modello dati al processo vero, non a quello immaginato

Movimento II: Dal foglio Excel al primo cancello di fiducia

Hook: La decisione più netta della prima settimana di questo repository non è qualcosa che è stato costruito. È `TelaiInAttesaPage.jsx` rimossa per intero, in un solo commit, perché non rispecchiava più come Scattolini gestisce davvero quel pezzo di processo.

Obiettivo del post: Argomentare, con un esempio reale, che allineare il modello dati al processo effettivo — anche quando significa eliminare una funzionalità — è una decisione più delicata e più utile che aggiungerne una, e mostrare cosa vuol dire farlo con un criterio esplicito invece che a istinto.

Contenuto:

- Il commit `1d93810` ("rimuove la pagina Telai in Attesa di Ordine e tutta la funzionalità collegata") ha toccato il componente pagina, il suo modale, la rotta lazy in `App.jsx`, l'array di navigazione, un percorso `handleConvertToVehicle`, e tre punti distinti in `backupUtils.js` (`BACKUP_COLLECTIONS`, `SHEET_HEADERS`, `COLLECTION_LABELS`) — una funzionalità che si era radicata nell'esportatore di backup senza che nessun punto unico la tracciasse.
- Confronta questo con la sorte di `modalitaRitiroConsentite`, una restrizione per-veicolo sui metodi di ritiro ammessi, rimossa mesi dopo (5 luglio, commit Multi-fix `79ca704`, Task 1 in `ARCHITETTURA_SSV_MANAGER.md` §12): il campo è stato lasciato inerte nei documenti Firestore già esistenti invece di essere migrato via, per criterio esplicito — uno script di migrazione che tocca dati di produzione comporta più rischio di una colonna morta e ignorata.
- Lo stesso commit ha rimosso `getModalitaRitiroConsentite()` da `src/Utils/ritiroUtils.js`: eliminare una funzionalità significa eliminare anche il suo helper, non lasciare un export inutilizzato che il prossimo che legge il file deve interpretare da sé.
- Il documento di architettura esplicita il criterio ogni volta che un campo viene abbandonato: *"nessuna migrazione: i documenti esistenti mantengono il campo, semplicemente ignorato."* Ogni deprecazione in questo codice segue quel criterio — o una sua variante — prima di decidere se uno script di migrazione vale il rischio.
- Non ancora deciso, per scelta: la collection orfana `telai` in `firestore.rules` — ancora concessa, ancora mai referenziata in `src/`, lasciata deliberatamente come domanda aperta invece che risolta a intuito (§10 del documento di architettura: "da chiarire con il cliente").

Impatto sul processo aziendale: Prima, un pezzo di processo dismesso (la gestione separata dei telai in attesa ordine) restava comunque visibile e scrivibile nel sistema, creando il rischio concreto di due fonti di verità in disaccordo su un veicolo. Dopo, il modello dati rispecchia solo il processo che Scattolini segue davvero oggi: chi guarda il sistema non deve più chiedersi "questa sezione è ancora quella giusta da usare?"

Chiusura: Un codice che non elimina mai nulla non è disciplinato: sta solo accumulando. Qual è una funzionalità nel tuo sistema che tutti sanno essere superata da come lavorate davvero, ma che nessuno ha ancora rimosso?

Visivo: nessun diagramma — qui funziona meglio uno screenshot del diff. [SCREENSHOT DA INSERIRE: il diff di `git show 1d93810 --stat` in un editor, per rendere visivamente quanti file tocca una singola rimozione pulita]

Post 4 — Un solo punto di verità richiede un solo posto nel codice

Movimento II: Dal foglio Excel al primo cancello di fiducia

Hook: `App.jsx` aveva 2000 righe e faceva sette lavori scollegati contemporaneamente. Un commit lo ha portato a 300 — senza cambiare un solo comportamento visibile.

Obiettivo del post: Ripercorrere un refactor reale, in un solo commit (`4373815`), come modello di scomposizione *sicura*: separare le responsabilità senza toccare il contratto che ciascun pezzo espone — perché un punto di verità unico, per reggere, deve essere unico anche nella struttura del codice, non solo nell'interfaccia che l'utente vede.

Contenuto:

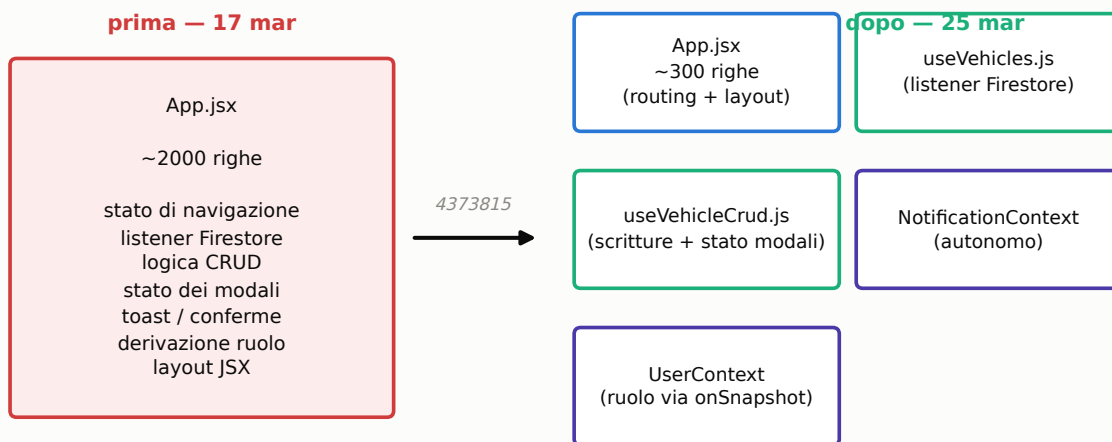
- Il commit `4373815` ("refactor: scompone App.jsx in hook mirati e context autonomi") ha estratto quattro responsabilità da `App.jsx` in un solo passaggio:
`src/hooks/useVehicles.js` (l'unico listener Firestore globale sulla collection `veicoli`),
`src/hooks/useVehicleCrud.js` (ogni percorso di scrittura + stato dei modali), e ha reso `NotificationContext` / `UserContext` autonomi invece di ricevere stato dall'alto.
- Il messaggio di commit dichiara il vincolo che ha reso sicuro farlo in un solo passaggio: *"tutto il comportamento realtime di Firestore, la logica RBAC e le interfacce dei componenti restano invariate."* Diff netto: `+603 / -542` su 5 file — quasi in pareggio, perché è uno spostamento, non una riscrittura.
- Cosa resta in `App.jsx` dopo: lo stato di navigazione (`currentPage`), lo stato del menu mobile, e il layout JSX — l'unica cosa per cui questo sistema non ha davvero un'astrazione, perché **non esiste un router** (confermato in `ARCHITETTURA_SSV_MANAGER.md` §2: la navigazione è un solo switch su `useState` , nessun link profondo, il refresh riporta sempre a `officina`). Criterio: introdurre un router sarebbe stato fuori scopo per questo refactor, che doveva isolare responsabilità esistenti, non aggiungerne di nuove.
- Il risultato è il pattern documentato al §8 del documento di architettura: un solo array `vehicles` globale, alimentato da un solo listener, passato via props a nove pagine — nessun React Query, nessun Redux, nessuna fetch per pagina. Il refactor ha reso quel pattern *visibile* nella struttura dei file invece che sepolto in un monolite.
- Questo è anche il file in cui è arrivato in seguito `useFilteredVehicles` (commit `30647fc` , task di performance) — un filtro per stato avvolto in `useMemo` che ha sostituito chiamate ripetute e non memoizzate `vehicles.filter(v => v.status === ...)` in quattro pagine. La decomposizione ha reso possibile quel passo successivo: sul file da 2000 righe sarebbe stato un diff molto più grande e più rischioso.

Impatto sul processo aziendale: Prima, aggiungere una regola di business o correggere un comportamento significava intervenire in un unico file da 2000 righe con sette responsabilità intrecciate — ogni modifica rischiava di toccare, per errore, un pezzo non correlato. Dopo, ogni responsabilità vive nel proprio modulo: chi lavora sul flusso di notifiche non deve leggere la logica di navigazione, e viceversa. Il costo di manutenzione futura scende in modo diretto e misurabile.

Chiusura: Il refactor sicuro da spedire in un solo commit è quello in cui puoi scrivere, onestamente, "le interfacce non sono cambiate". Qualunque cosa più rischiosa probabilmente non dovrebbe essere un commit solo.

Visivo: diagramma prima/dopo delle cinque responsabilità estratte da `App.jsx`.

Un solo punto di verità nel codice, non solo nell'interfaccia



Post 5 — La macchina a stati che prima viveva nel coordinamento a voce

Movimento III: Il sistema prende decisioni al posto delle persone

Hook: `veicoli/{id}.status` ha quattro valori. Passare dall'uno all'altro fa scattare in silenzio un'email, libera un posto auto, o chiude automaticamente una pratica di ritiro — passaggi che prima significavano una telefonata tra officina, ufficio e cliente.

Obiettivo del post: Rendere esplicite le regole di business che prima vivevano solo nella testa di chi coordinava reparti diversi via email e telefono. `ARCHITETTURA_SSV_MANAGER.md` §6 apre proprio con "queste sono sparse nel codice e facili da rompere involontariamente" — questo post trasforma quell'avviso in un diagramma che chiunque in azienda può controllare, tecnico o no.

Contenuto:

- `da-allestire → in-allestimento → pronto → ritirato` : la transizione a `pronto` compila da sé `dataMontaggio` con la data odierna se `modalitaRitiro !== 'montaggio'` — implementata in due punti indipendenti, `OfficinaPage.handleStatusChange` e `useVehicleCrud.handleSaveVehicle`. Il documento di architettura segnala per nome questa duplicazione come un punto da tenere d'occhio per non farla scattare due volte: il criterio è documentare il rischio dove vive, non fingere che non esista.
- La stessa transizione `in-allestimento → pronto` fa partire anche `POST /api/sendVehicleReady` — un'email vera a un cliente vero. Prima di questo automatismo, avvisare il cliente che il veicolo era pronto dipendeva da chi si ricordava di scrivere l'email; ora è una conseguenza diretta del cambio di stato nel sistema.
- `status → ritirato` libera il posto auto tramite `releaseParkingSpotForVehicle()` — un helper introdotto proprio per chiudere un buco reale nel coordinamento: prima del multi-fix del 5 luglio (`79ca704` , Task 4), il quick-toggle "Ritiro/Consegna svolto" in `PianificazioneRitiriPage` scriveva `status: 'ritirato'` direttamente su Firestore, bypassando l'unico punto che liberava il posto. Un veicolo segnato "fatto" da quella scorciatoia continuava a occupare, sulla carta, un posto che nella realtà era già libero — risolto estraendo la logica di rilascio in un helper condiviso, richiamato da entrambi i percorsi di scrittura.
- Un quarto automatismo gira su un orologio, non su un click: `scripts/autocompleteMontaggiAppuntamenti.js` , uno script Node autonomo con `firebase-admin` , eseguito da GitHub Actions ogni 15 minuti (`.github/workflows/auto-complete-montaggi.yml`), segna da sé un veicolo `ritirato` quando l'appuntamento di montaggio in loco è scaduto. Prima era un `useEffect` che riscansionava tutti i veicoli a ogni render di `PianificazioneRitiriPage` — spostato lato server con un criterio

preciso: un controllo temporale non deve dipendere dal fatto che qualcuno abbia quella pagina aperta nel browser.

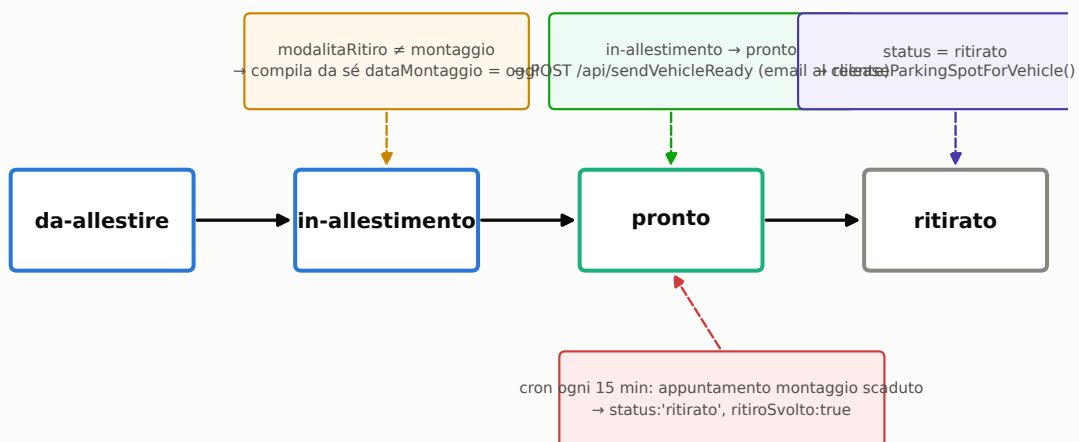
- Le azioni compiute dal cron vengono registrate nella collection `actions` con `userName: 'Sistema (automatico)'`, non con il nome di chi per caso aveva la pagina aperta — una scelta di tracciabilità precisa una volta che la logica si è spostata fuori dal client: il registro azioni deve distinguere una decisione automatica da una umana.

Impatto sul processo aziendale: Prima, sapere se un veicolo era stato davvero ritirato, se il posto auto era libero, o se il cliente era stato avvisato richiedeva ricostruire la situazione da email e memoria — con un margine di errore reale (posti auto segnati occupati quando non lo erano più). Dopo, questi passaggi sono conseguenze automatiche di un singolo cambio di stato: l'email parte da sola, il posto si libera da solo, la pratica si chiude da sola quando scade l'appuntamento — nessuno step dipende più dalla memoria di una persona specifica.

Chiusura: Se un automatismo si scopre solo leggendo il codice che lo attiva, non è documentato: sta solo aspettando di sorprendere la prossima persona che tocca quel file. Cosa rivelerebbe il tuo diagramma degli stati, se lo disegnassi davvero?

Visivo: i quattro stati con tutti e quattro gli automatismi annotati nel punto in cui scattano.

veicoli/{id}.status — quattro stati, quattro automatismi al posto del coordinamento a voce



Post 6 — Un sistema unico, ruoli su misura per chi tocca solo un pezzo del processo

Movimento III: Il sistema prende decisioni al posto delle persone

Hook: Il memo di gennaio all'amministrazione segnalava l'RBAC come un vuoto da colmare. A fine marzo, il sistema aveva già quattro ruoli — uno costruito su misura per un singolo interlocutore esterno che deve vedere quasi nulla.

Obiettivo del post: Mostrare l'RBAC non come un singolo commit "aggiungi ruoli", ma come qualcosa cresciuto in risposta a un bisogno reale e nominato — un ruolo costruito per un *omologatore* (chi valuta l'omologazione dell'allestimento), non un generico "utente in sola lettura".

Contenuto:

- L'RBAC di base — `admin` / `operativo` / `non_operativo` — era già applicato lato server in `firestore.rules` al primissimo commit (`canWrite()` restituisce falso per `non_operativo` ; `isAdmin()` protegge un elenco fisso di otto campi dealer/Ford tramite `adminFields()`). Qualunque cosa abbia chiuso il gap segnalato dal memo di gennaio è accaduta prima dell'inizio di questa storia git — un confine che questo documento non può attraversare.
- Il commit `c7c170c` ("aggiungi ruolo Omologatore con accesso limitato a Riepilogo", 26 marzo) ha aggiunto un quarto ruolo costruito su misura: sola lettura, menu ridotto a una sola pagina (`riepilogo`), barra di ricerca universale nascosta, un badge ruolo viola distinto in navigazione. Non un interruttore di permesso su un ruolo esistente — un ruolo nuovo, perché la forma di accesso era genuinamente diversa: chi valuta l'omologazione ha bisogno di una vista di sintesi, non degli strumenti operativi di officina.
- `firestore.rules` non ha mai fatto crescere una whitelist per singolo campo oltre agli otto `adminFields()` — verificato esplicitamente sia durante la feature del 26 marzo sia durante quella del 5 luglio per l'assegnazione da calendario: qualunque nuovo campo non-admin su un documento veicolo è automaticamente scrivibile da `operativo` / `admin` senza toccare le regole, per scelta. Il confine di fiducia è basato sul ruolo, non campo per campo — un criterio deliberato che evita di dover aggiornare le regole a ogni nuovo campo del form veicolo.
- La feature di assegnazione da calendario (§11 del documento di architettura) ha dovuto riverificare esplicitamente questo confine prima di essere rilasciata: due nuovi campi (`collaudo: 'pianificato'` , `statoRitiro`) sono stati controllati contro `adminFields()` e confermati dentro il perimetro di fiducia non-admin esistente — nessun deploy delle regole necessario per quella feature.

- Cosa l'RBAC non copre ancora, per scelta esplicita: non esiste un'interfaccia per gestire i ruoli. `users/{uid}.role` viene impostato da un admin direttamente nella console Firestore (confermato al §5: *"il ruolo va promosso manualmente da un admin via Firestore console, non c'è UI per gestire i ruoli"*) — un vuoto reale e riconosciuto, non una svista che nessuno ha notato.

Impatto sul processo aziendale: Prima, dare visibilità a un interlocutore esterno come l'omologatore avrebbe richiesto un estratto separato (un file Excel filtrato, un'email con lo stato aggiornato a mano) — un quarto pezzo di frammentazione accanto a Excel, email e SAP. Dopo, quello stesso interlocutore accede allo stesso sistema unico degli operatori interni, con una vista filtrata e in sola lettura costruita sul suo bisogno specifico: un solo punto di verità, letto con permessi diversi invece che duplicato in un altro formato.

Chiusura: Un ruolo non è davvero "utente in sola lettura" finché non sai dire per chi è pensato e cosa specificamente non deve vedere. Qual è l'ultimo ruolo che hai aggiunto, e potresti dire lo stesso?

Visivo: quattro ruoli confrontati su sei confini di permesso reali, presi direttamente da `firestore.rules` e `UserContext.jsx`.

Un sistema unico, accessi su misura per chi tocca solo un pezzo del processo

`firestore.rules`: `canWrite()` / `isAdmin()` / `adminFields()` — la UI client rispecchia le stesse regole

	admin	operativo	non_operativo	omologatore
Lettura veicoli	✓	✓	✓	✓
Scrittura veicoli	✓	✓	✗	✗
Campi Admin (dealer / prezzo Ford)	✓	✗	✗	✗
config/* (destinatari email)	✓	✗	✗	✗
Pagina Backup Locale	✓	✗	✗	✗
Nav completa (9 pagine)	✓	✓	✓	✗

Post 7 — Automatizzare la comunicazione al cliente senza legarsi a un costo

Movimento III: Il sistema prende decisioni al posto delle persone

Hook: L'email automatica "veicolo pronto" era una Cloud Function Firebase. È stata tolta dalla produzione — non perché si fosse rotta, ma perché tenerla in piedi avrebbe richiesto un piano a pagamento che il progetto ha scelto di non sottoscrivere.

Obiettivo del post: Una decisione di architettura reale, guidata da costo e semplicità operativa e non da un guasto tecnico — con la documentazione che conferma questo ordine dei fatti.

Contenuto:

- Il commit `ae6dfa3` (23 aprile) ha introdotto per primo l'email automatica come una vera Cloud Function v2: trigger `onDocumentUpdated` su `veicoli`, logica di retry, template HTML, destinatari configurabili via `config/emailNotifications` — un'implementazione reale e funzionante, il primo passo per eliminare l'avviso manuale al cliente.
- Il commit `d5cb503` (22 maggio, "sostituisce Cloud Functions con Vercel API + GitHub Actions cron") l'ha sostituita un mese dopo: il trigger è diventato `api/sendVehicleReady.js`, una funzione serverless Vercel chiamata lato client subito dopo la scrittura dello stato, autenticata con un token Firebase ID; il report settimanale COC/pagamenti è diventato uno script Node autonomo (`scripts/sendWeeklyReport.js`) eseguito da un cron GitHub Actions (`weekly-report.yml` , lunedì 07:00 UTC). Il messaggio di commit dichiara il criterio in modo diretto: *"Elimina la dipendenza da Firebase Cloud Functions (piano Blaze non necessario)."* Criterio di scelta: automatizzare la comunicazione al cliente non deve dipendere da un piano a consumo che introduce un vincolo di costo evitabile.
- Il vecchio codice non è stato cancellato: è stato spostato in `_deprecated/functions/` con un proprio `README.md` che spiega esattamente perché è inerte — `.github/workflows/deploy-functions.yml` (nonostante il nome) deploya solo `firestore:rules`, mai `functions/`, quindi la Cloud Function non era più viva in produzione da un po', prima ancora che qualcuno la marcasse formalmente come tale.
- La migrazione ha prodotto una regressione reale, individuata e corretta in seguito: quando `functions/` è stata rinominata in `_deprecated/functions/`, i percorsi relativi dei template in `api/sendVehicleReady.js` e `scripts/sendWeeklyReport.js` (più `includeFiles` in `vercel.json`) continuavano a puntare al vecchio percorso, ormai inesistente — ogni invio email falliva in silenzio con `ENOENT` finché il commit `c129946`

non ha corretto i percorsi. Un promemoria concreto: anche una migrazione di infrastruttura ben motivata va verificata end-to-end, non solo nella sua logica di principio.

- Entrambi i sistemi attivi portano oggi lo stesso commento in testa — `// SISTEMA EMAIL ATTIVO — modificare qui` — una protezione a bassa tecnologia ma efficace contro chi in futuro corregga un bug nella copia deprecata e si chieda perché nulla cambia.

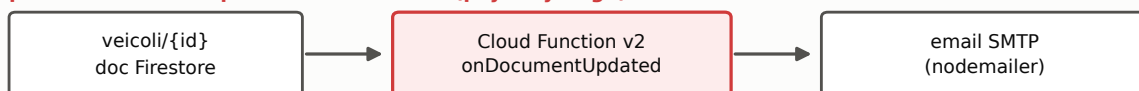
Impatto sul processo aziendale: Prima dell'automatismo, avvisare il cliente che il veicolo era pronto dipendeva da chi si ricordava di farlo, con tempi variabili. Dopo la prima automazione (Cloud Function), il passaggio era già automatico ma vincolato a un piano di costo non necessario per le dimensioni del progetto. Dopo la migrazione a Vercel + GitHub Actions, la stessa comunicazione resta automatica e verificabile, senza legare un'operazione di routine a una soglia di spesa che avrebbe dovuto essere approvata e monitorata separatamente.

Chiusura: La migrazione di architettura più economica è quella fatta prima che il piano costoso venga mai attivato, non dopo che arriva la fattura. Qual è, nel tuo stack, una dipendenza a un passo da un cambio di fascia di prezzo che potrebbe diventare una scelta forzata?

Visivo: diagramma prima/dopo dell'architettura: trigger Cloud Function contro API Vercel chiamata dal client + cron.

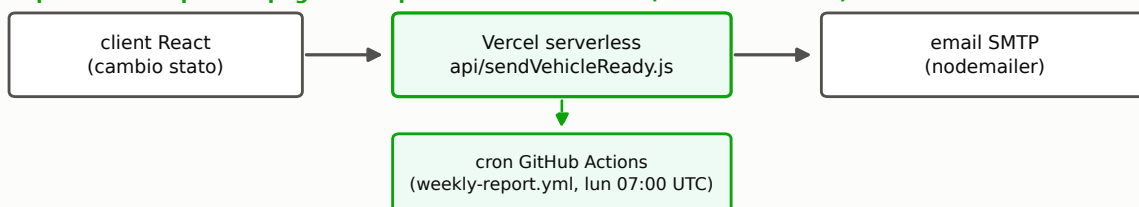
Automatizzare la comunicazione al cliente senza legarsi a un costo

prima — richiede il piano Firebase Blaze (pay-as-you-go)



stato: _deprecated/functions/ — non deployato da nessun workflow attivo

dopo — nessun piano a pagamento per le Cloud Functions (commit d5cb503)



Post 8 — Proteggere il punto di verità unico: il backup che si fermava in silenzio

Movimento III: Il sistema prende decisioni al posto delle persone

Hook: Questa settimana, un cambio di progetto Firebase ha fatto fallire in silenzio ogni backup programmato — e il criterio corretto, verificato subito, è stato tornare al progetto precedente prima che il problema diventasse invisibile più a lungo.

Obiettivo del post: Un incidente reale e datato (3 luglio 2026) raccontato come verifica operativa di una garanzia che il sistema unico non diventi, a sua insaputa, un unico punto di fallimento — il post più "da founder" della serie, perché mostra un criterio applicato sotto pressione, non solo in fase di progettazione.

Contenuto:

- Il commit `30c4671` (3 luglio, "revert(firebase): torna al progetto scattolini-6143a (cf17b senza billing)") documenta la sequenza con precisione: il progetto Firebase attivo era stato spostato su `scattolini-ssv-manager-cf17b`, e quel progetto si è rivelato sul piano **Spark (gratuito)** di Firebase, che blocca l'operazione di `export` Firestore da cui dipende il workflow di backup giornaliero. Il criterio di verifica è stato applicato subito: un backup che sembra configurato correttamente ma non può eseguire l'operazione che promette non è un backup, è un falso senso di sicurezza.
- `scattolini-6143a` — il progetto originale, presente in `firestore-backup.yml` / `storage-backup.yml` fin dal primo commit — aveva il billing attivo per tutto il periodo. Il commit di revert è esplicito sul fatto che **non serviva alcuna migrazione dati**: quanto era stato scritto su `cf17b` nel frattempo andava scartato e reinserito, non migrato — criterio: quando il volume di dati nel periodo di transizione è piccolo e noto, ripartire pulito costa meno che costruire una migrazione per un ambiente mai stato la fonte di verità di produzione.
- La correzione ha toccato più di `.firebaseerc`: `firestore-backup.yml`, `storage-backup.yml` (percorso del bucket ripristinato da `git show` del commit che l'aveva cambiato, non ridigitato a memoria), e `deploy-functions.yml`, esteso nello stesso commit per deployare finalmente anche `firestore:indexes` e non solo `firestore:rules` — un gap presente fin dal primo commit, individuato mentre si verificava qualcos'altro.
- Il messaggio di commit dichiara con precisione cosa non era verificabile da quell'ambiente: se il segreto `GCP_SA_KEY` corrisponda ancora al progetto ripristinato, e cosa sia realmente attivo oggi su `scattolini-6143a` — entrambi segnalati esplicitamente invece che dati per scontati, perché un criterio di verifica onesto include dichiarare cosa non si può ancora confermare.

- **[DA CONFERMARE: se il backup automatico giornaliero (`firestore-backup.yml` , cron `0 2 * * *`) ha effettivamente ripreso a funzionare dopo il revert — verificabile solo dai log di GitHub Actions, non dal contenuto del repository]**

Impatto sul processo aziendale: Prima di questa verifica, l'azienda avrebbe potuto scoprire l'assenza di backup solo nel momento peggiore possibile: durante un ripristino necessario. Il criterio applicato qui — verificare che l'infrastruttura di backup funzioni davvero, non solo che sia configurata — ha trasformato un rischio silenzioso e potenzialmente enorme (perdita di dati aziendali su ordini, allestimenti, pagamenti) in un aggiustamento di configurazione risolto nella stessa giornata.

Chiusura: Un backup che fallisce in silenzio è peggio di uno che fallisce rumorosamente: te ne accorgi solo al momento del ripristino. Qual è, nel tuo sistema, il job automatico che dai per riuscito solo perché non si è ancora lamentato?

Visivo: nessun diagramma generato — post narrativo/di incidente, meglio servito da [SCREENSHOT DA INSERIRE: la tab "Actions" di GitHub con la cronologia delle esecuzioni del workflow `firestore-backup.yml` , per mostrare visivamente le esecuzioni fallite prima del revert e quelle riuscite dopo].

Post 9 — Un solo criterio di ricerca, zero conflitti tra operatori

Movimento IV: Igiene di produzione e chiusura del cerchio

Hook: Otto pagine diverse avevano ciascuna una propria idea di cosa significasse "cercare un veicolo". Nessuna corrispondeva alle altre, e nessuna era completa.

Obiettivo del post: Due interventi distinti ma della stessa settimana (entrambi nel commit `79ca704`, 5 luglio) che si leggono come un unico criterio: portare a un solo punto ciò che continuava a essere duplicato, e far sì che due operatori che lavorano sullo stesso veicolo nello stesso momento vengano arbitrati dal sistema, non lasciati a scoprirlo a voce in officina.

Contenuto:

- Prima: la barra di ricerca dell'header, sei pagine, e `VehiclePickerList` avevano ciascuna una propria lista di campi scritta a mano — `VehiclePickerList` cercava su 3 campi (numero telaio, matricola, committente), `OfficinaPage` su un sottoinsieme diverso da quello di `RiepilogoPage`. Nessuna delle otto liste era allineata alle altre né allo schema reale del veicolo: un operatore poteva cercare un veicolo per un campo che una pagina copriva e un'altra no.
- Dopo: `src/utils/searchUtils.js` espone `searchVehicle()` / `filterVehiclesBySearch()` — una sola funzione che scorre dinamicamente ogni chiave valorizzata dell'oggetto veicolo (esclude solo l'id del documento Firestore), scendendo in valori annidati come `clienteAvvisato.data` e nei nomi dei file dentro `files[]` / `distinta` fino a due livelli di profondità. Aggiungere un campo allo schema veicolo lo rende ora cercabile ovunque, senza scrivere altro codice — il criterio scelto è che la ricerca deve essere una proprietà del sistema, non una lista da mantenere manualmente in sincrono in otto punti diversi.
- Le prestazioni sono state affrontate esattamente nel punto in cui sarebbero regredite: la ricerca resta interamente client-side sull'unico array `vehicles` già tenuto in memoria da `useVehicles()` — nessuna nuova query Firestore per ogni carattere digitato — ma il testo cercabile di ogni veicolo viene memoizzato in una `WeakMap` con chiave l'oggetto veicolo stesso, così i tasti successivi contro un array `vehicles` invariato costano un `String.includes()`, non una nuova scansione completa di ogni chiave di ogni veicolo.
- Separatamente, la feature di assegnazione da calendario della stessa sessione (§11 del documento di architettura, rilasciata un giorno prima) avvolge ogni scrittura di assegnazione rapida in una `runTransaction` di Firestore: rilegge il veicolo dentro la transazione e verifica che sia *ancora* nel gruppo assegnabile (`collaudo === 'da-`

`collaudare' && !dataCollaudo` per i collaudi; `status === 'pronto' && !dataRitiro` per i ritiri) prima di scrivere. Se un secondo operatore ha assegnato lo stesso veicolo per primo, la transazione fallisce e chi ha agito per secondo riceve un errore visibile — non una sovrascrittura silenziosa del lavoro di un collega.

- La vecchia `searchVehicle()` in `validationUtils.js` (una lista fissa di circa 20 campi, priva di `posizioneParcheggio`, `codiceInventario`, `cocFase1`, `ritiroGiorno` e di ogni campo admin non-dealer) è stata rimossa del tutto una volta migrato il suo unico consumatore rimasto (`UniversalSearch.jsx`) — nessuna implementazione parallela lasciata a metà.

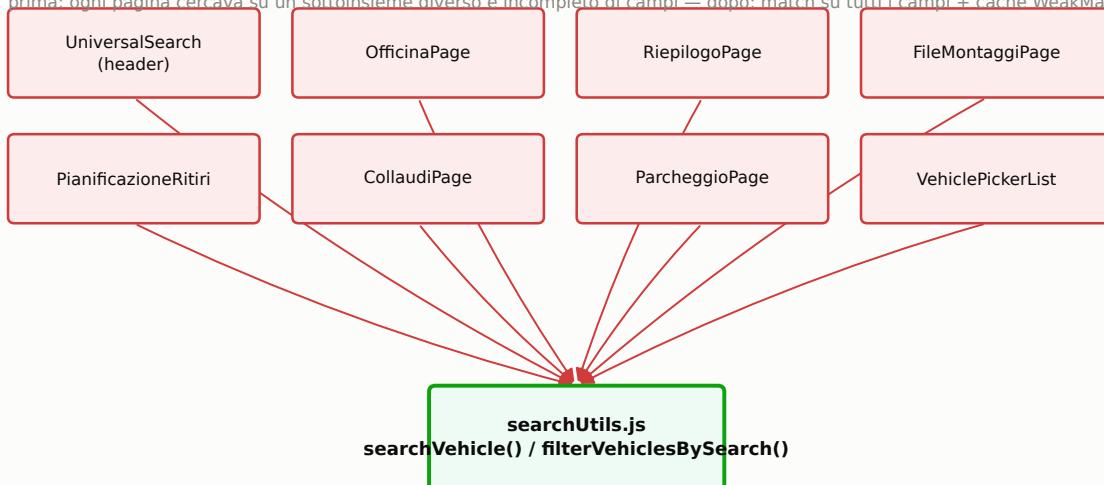
Impatto sul processo aziendale: Prima, trovare un veicolo dipendeva da quale pagina si stava usando: un campo cercabile in Officina poteva non esserlo in Riepilogo, costringendo l'operatore a "sapere" in quale sezione cercare cosa. Prima, inoltre, due persone potevano assegnare lo stesso veicolo a un collaudo o a un ritiro nello stesso momento senza che il sistema se ne accorgesse, scoprendo il conflitto solo dopo, a voce. Dopo, la ricerca è identica ovunque nel sistema, e un'assegnazione concorrente viene bloccata e segnalata nel momento stesso in cui accade — il coordinamento tra operatori passa dal sistema, non dalla memoria di chi ha parlato con chi.

Chiusura: La duplicazione nella logica di ricerca resta innocua finché lo schema non cambia, e sette delle tue otto copie diventano silenziosamente obsolete. Dove, nel tuo sistema, esiste già "lo stesso filtro, scritto otto volte leggermente diverso"?

Visivo: gli otto punti di ricerca sparsi che confluiscono in un unico helper condiviso.

Un solo criterio di ricerca, uguale in ogni pagina

prima: ogni pagina cercava su un sottoinsieme diverso e incompleto di campi — dopo: match su tutti i campi + cache WeakMap per veicolo



Post 10 — Ripulire la casa: quando il sistema non ha più bisogno delle sue impalcature

Movimento IV: Igiene di produzione e chiusura del cerchio — con un bilancio finale

Hook: Nella stessa settimana in cui i backup di questo sistema sono stati verificati e la ricerca è stata unificata in un solo criterio, tre documenti a uso esclusivamente interno sono stati eliminati definitivamente dal repository. Non è trascuratezza: è cosa succede quando un sistema smette di essere "in valutazione" e diventa lo strumento che l'azienda usa ogni giorno.

Obiettivo del post: Chiudere la serie dove si è aperta — con i documenti, non con il codice — usando la loro rimozione come segnale che il progetto ha smesso di essere una proposta sotto esame ed è diventato un sistema di produzione. E tirare le somme, con lo stesso criterio usato in ogni post: quale pezzo del processo frammentato è stato assorbito, e a quale prezzo.

Contenuto:

- 5 luglio: `README.md` (il README generico e mai personalizzato del template Vite, presente fin dal primo commit), `VERCEL_SETUP.md`, e `DOCUMENTO_TECNICO_AMMINISTRAZIONE.md` — la proposta di gennaio con cui si è aperta questa serie — sono stati eliminati nella stessa sessione di pulizia (`3348696`, `f9ddd42`, `eb9aba7`). Il documento di proposta aveva esaurito il suo scopo: non doveva più convivere con il codice sorgente una volta che la decisione che argomentava era già stata presa e messa in pratica.
- La stessa sessione (`dee0fb7`, "chore(cleanup): rimuove file morti e duplicati, aggiorna .gitignore") ha anche rinominato `functions/` in `_deprecated/functions/`, il che ha rotto due percorsi relativi ai template e ha richiesto una correzione un commit dopo (`c129946`) — un promemoria operativo: anche un commit "di sola pulizia" può far regredire un percorso di codice attivo se tocca una cartella referenziata altrove con un percorso relativo, quindi ogni pulizia va verificata end-to-end come una modifica qualunque.
- `ARCHITETTURA_SSV_MANAGER.md` (61 KB, mantenuto — a differenza dei documenti eliminati) è esso stesso un artefatto di igiene: ogni sessione aggiunge una sezione numerata che documenta cosa è stato verificato, cosa è emerso, e cosa è stato deliberatamente *non* cambiato e perché — inclusa una lista aggiornata del debito tecnico noto (§10: nessun test, nessun router, nessuna paginazione sul listener dei veicoli, la collection orfana `telai`). Il criterio è lo stesso di tutta la serie: il debito tecnico va scritto e tracciato, non taciuto.

- Sull'intera storia: 84 dei 141 commit portano un trailer `Co-Authored-By: Claude` e un link `claude.ai/code/session_...`; gli altri 57 sono per lo più i 45 commit di merge più un piccolo numero di interventi diretti (modifiche ai workflow YAML, caricamenti di file, cancellazioni) fatti dall'account umano. Questa è la forma concreta di come il sistema è stato costruito: decisioni prese e verificate sessione dopo sessione, non un unico sprint. **[DA CONFERMARE: il ruolo esatto della persona dietro l'account Scattolini2026 nel decidere cosa costruire — il repository mostra *quali* commit sono umani, non le conversazioni che li hanno motivati]**
- Non esiste una suite di test in questo repository (`package.json` non ha uno script di test; confermato al §1 del documento di architettura: "non c'è test runner configurato"). Cosa lo compensa, in modo imperfetto ma reale: `npm run lint` e `npm run build` eseguiti dopo ogni sessione, e un'abitudine di verifica-e-referto — il Task 5 del documento di architettura, ad esempio, ha ricostruito manualmente un file di export, riaperto con la libreria `xlsx`, e confrontato colonna per colonna con lo schema prima di dichiarare l'export "verificato". Il criterio applicato ovunque in questa serie, reso esplicito qui per la prima volta: in assenza di una rete di test automatica, ogni claim tecnico va verificato a mano, con lo stesso rigore, prima di essere considerato vero.

Impatto sul processo aziendale: Prima, il processo di Scattolini per gestire un veicolo allestito viveva diviso in tre sistemi — Excel per il tracciamento, email per il coordinamento tra reparti e con il cliente, SAP per gli ordini — nessuno dei quali era il punto di verità, e la proposta di unificarli era un documento isolato in attesa di approvazione. Dopo quattro mesi documentati in 141 commit, quello stesso processo vive in un sistema unico con un solo schema dati, ruoli su misura per chi vi accede, automatismi che sostituiscono il coordinamento a voce, e backup verificati — e i documenti che giustificavano la proposta iniziale non servono più, perché il sistema stesso è la prova che funziona.

Chiusura: La versione di "pronto per la produzione" che compare in una slide e quella che compare in una storia di commit sono due documenti diversi — questa serie è stata scritta a partire dal secondo. Cosa racconterebbe la storia commit del tuo prodotto, se qualcuno la leggesse dall'inizio alla fine come è stata letta questa?

Visivo: nessun diagramma generato — post di chiusura, guidato dal testo. [SCREENSHOT DA INSERIRE: la sezione finale di `ARCHITETTURA_SSV_MANAGER.md` (§10, "Punti di attenzione / debito tecnico"), per mostrare che il debito tecnico è scritto e tracciato, non nascosto]

Piano di pubblicazione

CADENZA CONSIGLIATA

Due post a settimana, martedì e giovedì — abbastanza distanziati perché ogni post venga letto per sé, non così tanto da perdere il filo della serie (cinque settimane in totale per dieci post). Ogni post riprende lo stesso elemento grafico di serie (`docs/assets/timeline.svg`, ritagliato per evidenziare la finestra temporale di quel post), così la serie è riconoscibile nel feed prima ancora di leggere il testo.

ORDINE DI PUBBLICAZIONE CONSIGLIATO (NON CRONOLOGICO)

L'ordine cronologico seppellisce gli hook più forti a metà serie. Si parte dai due post più controintuitivi, si alternano profondità tecnica e narrazione, e si chiude con il bilancio finale:

1. **Post 1** — Il memo pre-repository (hook più forte: "la storia inizia prima del codice")
2. **Post 8** — L'incidente sui backup (secondo hook più forte: attualità + onestà del criterio; funziona anche da solo se la serie viene interrotta)
3. **Post 5** — La macchina a stati mai disegnata esplicitamente (diagramma più condivisibile)
4. **Post 6** — Chiudere il gap RBAC (riprende direttamente il memo del Post 1)
5. **Post 2** — Da zero al cancello di fiducia sui dati
6. **Post 7** — Sostituire le Cloud Functions prima della fattura
7. **Post 4** — Scomporre il monolite
8. **Post 3** — Allineare il modello dati al processo vero
9. **Post 9** — Un solo criterio di ricerca / la corsa critica evitata
10. **Post 10** — Pulizia come segno di maturità + bilancio finale (deve restare ultimo indipendentemente dal resto dell'ordine: cita il Post 1 per nome e richiede quel contesto nel lettore)

CHECKLIST PRE-PUBBLICAZIONE (PER OGNI POST)

- ☐ Ogni percorso file, hash di commit e nome funzione nel post riverificato con `git show <hash>` appena prima della pubblicazione (il repository può essere andato avanti da quando è stato scritto questo documento).
- ☐ Nessun marker `[DA CONFERMARE: ...]` lasciato irrisolto nel testo pubblicato — o confermare il fatto con chi ha accesso alle dashboard Firebase/Vercel, o tagliare l'affermazione, o lasciare la parentesi visibile se il post è esplicitamente inquadrato come "ecco cosa non posso verificare".
- ☐ Ogni `[SCREENSHOT DA INSERIRE: ...]` sostituito con un'immagine reale, o la frase che lo riferimento rimossa — nessun testo segnaposto esce in un post pubblicato.

- ☐ Nessun nome cliente, codice dealer, numero di telaio, prezzo, o qualunque cosa dalla lista `adminFields()` compare in uno screenshot (riverificare i *pixel effettivi*, non solo la didascalia — una scheda del browser sullo sfondo o uno screenshot della console Firestore possono far trapelare più di quanto previsto).
- ☐ Rileggere il post come se lo leggesse qualcuno in Scattolini non tecnico — confermare che nulla suoni come una lamentela su una decisione presa da un collega in precedenza (es. il revert del progetto `cf17b`, la regressione `ENOENT` sulle email) piuttosto che come una lezione appresa.
- ☐ Confermare che la chiusura del post sia una domanda, non una call-to-action travestita.